



SVEUČILIŠTE JOSIPA JURJA STROSSMAYERA U OSIJEKU

ODJEL ZA MATEMATIKU

Sveučilišni diplomski studij matematike
smjer: Matematika i računarstvo

Reed-Solomonovi kodovi za ispravljanje pogrešaka

SEMINARSKI RAD

Jurica Jurčević

Osijek, 2026.

Sadržaj

1	Uvod	1
2	Povijesni razvoj	3
3	Matematička podloga	5
3.1	Konačna polja	5
3.2	Polinomi nad poljem	6
4	Definicija kôda i kodiranje	7
4.1	Parametri kôda	7
4.2	Generirajući polinom	7
4.3	Izvorna konstrukcija: kôd kao vrijednosti polinoma	9
5	Dekodiranje	11
5.1	Sindromi	11
5.2	Polinom lokatora pogrešaka	11
5.3	Berlekamp-Masseyev algoritam	12
5.4	Chienova pretraga	13
5.5	Forneyev algoritam	13
5.6	Primjer nad većim poljem	14
5.7	Brisanja	14
5.8	Dekodiranje popisom	14
6	Primjene	17
7	Zaključak	19
	Literatura	21
	Sažetak	23
	Summary	25

1 | Uvod

Podatak koji putuje kroz stvarni kanal rijetko stigne netaknut. Grebotina na CD-u, oštećen dio QR koda, šum na radijskoj vezi, sve to mijenja pojedine bitove. Kodovi za ispravljanje pogrešaka rješavaju taj problem tako da uz podatak pošalju dodatne simbole iz kojih se izvorni podatak može rekonstruirati, čak i kad dio primljenog niza nije točan.

Reed-Solomonovi kodovi jedni su od najraširenijih takvih kodova. Irving Reed i Gustave Solomon opisali su ih 1960. godine u kratkom radu od pet stranica [1]. Ideja je algebarska. Podatak se shvati kao polinom, a kodna riječ nastaje iz vrijednosti tog polinoma. Dvije razne poruke daju polinome koji se podudaraju u malom broju točaka, pa se i njihove kodne riječi razlikuju na dovoljno mjesta da se pogreške mogu i otkriti i ispraviti.

Snaga tih kodova je u radu sa simbolima, a ne s pojedinačnim bitovima. Jedan simbol, primjerice jedan bajt, kôd tretira kao cjelinu, pa se oštećenje cijelog bajta broji kao jedna pogreška bez obzira na to koliko je bitova u njemu promijenjeno. Zato su dobri protiv rafalnih pogrešaka, onih koje pogode niz uzastopnih bitova odjednom, kakve nastaju kod grebotine na disku.

Nakon kratkog povijesnog osvrt, rad kreće od potrebne algebre, konačnih polja i polinoma nad njima. Zatim definira kôd, dokazuje njegovu minimalnu udaljenost i pokazuje kodiranje preko generirajućeg polinoma. Slijedi dekodiranje, koje je zahtjevniji dio, kroz sindrome, Berlekamp-Masseyev algoritam, Chienovu pretragu i Forneyev algoritam, uz zaseban odjeljak s rekurzijom i tablicom iteracija za Berlekamp-Masseyev korak. Svaki korak prati potpuno proveden brojčani primjer nad malim poljem. Nakon klasičnog dekodiranja dolaze dva proširenja koja se u praksi stalno koriste, dekodiranje uz brisanja i dekodiranje popisom koje prelazi klasičnu granicu ispravljlivosti. Na kraju su primjene. Svi brojčani rezultati u radu proizvedeni su priloženom implementacijom `kod/rs.py`.

2 | Povijesni razvoj

Irving S. Reed i Gustave Solomon opisali su kôd 1960. godine, radeći u Lincolnovu laboratoriju MIT-a [1]. Pristup im je bio algebarski, a opis koji ovaj rad koristi, izvornome ekvivalentan, ustalio se nešto kasnije.

Kôd je isprva bio ispred svog vremena jer učinkovit postupak dekodiranja nije postojao. To se promijenilo krajem šezdesetih, kada su Berlekamp (1968.) i Massey (1969.) dali algoritam koji i danas čini jezgru dekodera. Prvu veliku primjenu kôd je našao u svemirskom programu Voyager 1977. godine, a široku upotrebu donio je kompaktni disk 1982.; otad je posvuda gdje podaci prolaze nepozdanim kanalom [3].

3 | Matematička podloga

3.1 Konačna polja

Reed-Solomonovi kodovi rade nad konačnim poljem. Polje je skup na kojem su definirani zbrajanje, oduzimanje, množenje i dijeljenje s uobičajenim svojstvima, a konačno je ono s konačno mnogo elemenata. Za svaki prosti broj p i prirodan broj m postoji polje s p^m elemenata, jedinstveno do na izomorfizam, koje označavamo s $GF(p^m)$. Oznaka GF dolazi od engleskog *Galois field*, tj. Galoisovo polje, po Évaristeu Galoisu. U praksi je gotovo uvijek $p = 2$, jer se tada jedan simbol poklapa s nizom bitova. Polje $GF(2^8)$ ima 256 elemenata i svaki od njih stane u jedan bajt.

Definicija 1. Polje $GF(2^m)$ konstruiramo kao skup polinoma nad $GF(2)$ stupnja manjeg od m , uz zbrajanje i množenje po modulu nekog nesvodljivog polinoma stupnja m .

Zbrajanje u $GF(2^m)$ je zbrajanje polinoma nad $GF(2)$, što se svodi na operaciju *isključivo* ili po bitovima. Množenje je množenje polinoma, a zatim uzimanje ostatka po nesvodljivom polinomu. Svaki element različit od nule može se zapisati kao potencija jednog jedinog elementa α , koji zovemo primitivnim.

Primjer 1. Uzmimo $GF(8)$ s nesvodljivim polinomom $x^3 + x + 1$. Primitivni element je $\alpha = x$, pa vrijedi $\alpha^3 = \alpha + 1$. Uzastopnim množenjem dobivamo cijelu tablicu polja:

potencija	polinom	bitovi
α^0	1	001
α^1	x	010
α^2	x^2	100
α^3	$x + 1$	011
α^4	$x^2 + x$	110
α^5	$x^2 + x + 1$	111
α^6	$x^2 + 1$	101

Tablica 3.1: Elementi polja $GF(8)$ kao potencije primitivnog elementa α .

Iz tablice se lako množi. Umnožak $\alpha^i \cdot \alpha^j$ je $\alpha^{(i+j) \bmod 7}$, jer je $\alpha^7 = 1$. Tako se množenje svodi na zbrajanje eksponenata, a dijeljenje na oduzimanje. Priložena implementacija upravo ovako radi, s tablicom logaritama i tablicom potencija.

3.2 Polinomi nad poljem

Poruku duljine k simbola shvaćamo kao polinom

$$m(x) = m_{k-1}x^{k-1} + \cdots + m_1x + m_0,$$

gdje su koeficijenti m_i elementi polja $GF(2^m)$. Kodiranje i dekodiranje su operacije nad takvim polinomima. Ključna činjenica koju koristimo je da polinom stupnja d nad poljem ima najviše d nultočaka. Odatle slijedi razmak među kodnim riječima, koji omogućuje ispravljanje.

4 | Definicija kôda i kodiranje

4.1 Parametri kôda

Reed-Solomonov kôd označavamo s $RS(n, k)$. Ovdje je n duljina kodne riječi u simbolima, a k broj podatkovnih simbola. Za polje $GF(2^m)$ vrijedi $n = 2^m - 1$ za puni kôd, dok su kraće inačice dopuštene. Razlika $n - k$ je broj dodatnih, paritetnih simbola.

Teorem 1. *Minimalna udaljenost Reed-Solomonovog kôda $RS(n, k)$ iznosi $d = n - k + 1$. Kôd time dostiže Singletonovu granicu $d \leq n - k + 1$ s jednakošću, pa je optimalan u smislu razmaka (engl. maximum distance separable, MDS).*

Dokaz dajemo u odjeljku 4.2, nakon što uvedemo generirajući polinom, jer se oslanja na djeljivost svake kodne riječi njime. Kôd s minimalnom udaljenošću d ispravlja do

$$t = \left\lfloor \frac{d-1}{2} \right\rfloor = \left\lfloor \frac{n-k}{2} \right\rfloor$$

pogrešaka. Svaki dodatni paritetni simbol podiže sposobnost ispravljanja za pola pogreške. Dva paritetna simbola ispravlja jednu pogrešku, deset ih ispravlja pet.

4.2 Generirajući polinom

Kodne riječi definiramo preko generirajućeg polinoma

$$g(x) = \prod_{i=1}^{n-k} (x - \alpha^i), \quad (4.1)$$

čije su nultočke uzastopne potencije primitivnog elementa. Polinom $c(x)$ stupnja manjeg od n je kodna riječ točno onda kada je djeljiv s $g(x)$. Kako $g(x)$ ima $n - k$ nultočaka $\alpha^1, \dots, \alpha^{n-k}$, ekvivalentan je uvjet

$$c(\alpha^i) = 0, \quad i = 1, \dots, n - k. \quad (4.2)$$

Upravo na tih $n - k$ uzastopnih nultočaka počiva i minimalna udaljenost kôda. Sada možemo dokazati Teorem 1.

Dokaz Teorema 1. Za linearan kôd minimalna udaljenost jednaka je najmanjoj težini kodne riječi različite od nule, gdje je težina broj koeficijenata različitih od nule. Pretpostavimo suprotno, da postoji kodna riječ $c(x) \neq 0$ težine w s $1 \leq w \leq n - k$. Neka su njezini nenul koeficijenti $c_{j_1}, \dots, c_{j_w}$ na različitim pozicijama $j_1 < \dots < j_w$. Uzmemo li prvih w jednadžbi iz (4.2), dobivamo sustav

$$\sum_{l=1}^w c_{j_l} (\alpha^{j_l})^i = 0, \quad i = 1, \dots, w,$$

s matricom $A = [(\alpha^{j_l})^i]_{i,l}$. Izlučivanjem α^{j_l} iz l -tog stupca ostaje Vandermondeova matrica s čvorovima $\alpha^{j_1}, \dots, \alpha^{j_w}$, pa je

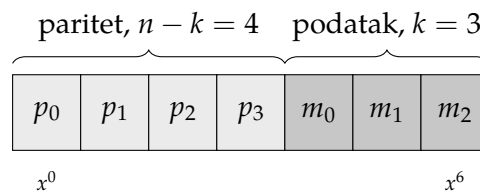
$$\det A = \left(\prod_{l=1}^w \alpha^{j_l} \right) \prod_{p < q} (\alpha^{j_q} - \alpha^{j_p}).$$

Potencije $\alpha^{j_1}, \dots, \alpha^{j_w}$ su međusobno različite jer α ima red n , a svi j_l leže u rasponu $0, \dots, n - 1$. Zato su svi činitelji različiti od nule, pa je $\det A \neq 0$. Sustav tada ima samo trivijalno rješenje $c_{j_1} = \dots = c_{j_w} = 0$, što je u suprotnosti s pretpostavkom da su to nenul koeficijenti. Dakle svaka kodna riječ različita od nule ima težinu barem $n - k + 1$, pa je $d \geq n - k + 1$. S druge strane, Singletonova granica $d \leq n - k + 1$ vrijedi za svaki (n, k) kôd [2, pogl. 1]. Slijedi $d = n - k + 1$. $\square$

Sistematsko kodiranje gradi kodnu riječ tako da k podatkovnih simbola ostane doslovno vidljivo, a $n - k$ paritetnih simbola zauzme niske potencije $x^0, \dots, x^{n-k-1}$. Raspored je prikazan na slici 4.1. Za poruku $m(x)$ računamo

$$c(x) = x^{n-k}m(x) - (x^{n-k}m(x) \bmod g(x)). \quad (4.3)$$

Prvi član pomiče podatak na visoke potencije $x^{n-k}, \dots, x^{n-1}$ i ostavlja niske potencije prazne. Drugi član je ostatak dijeljenja s $g(x)$, koji popuni to prazno mjesto paritetom; oduzimanjem ostatka razlika postaje djeljiva s $g(x)$, dakle kodna riječ. U polju karakteristike 2 oduzimanje i zbrajanje su ista operacija.



Slika 4.1: Raspored simbola u sistematskoj kodnoj riječi $RS(7, 3)$. Paritet zauzima niske potencije, podatak visoke.

Primjer 2. Za $RS(7, 3)$ nad $GF(8)$ imamo $n - k = 4$ paritetna simbola, pa kôd ispravlja $t = 2$ pogreške. Generirajući polinom prema (4.1) ispada

$$g(x) = x^4 + \alpha^3 x^3 + x^2 + \alpha^1 x + \alpha^3.$$

Uzmimo poruku $m(x) = x^2$. Formula (4.3) daje kodnu riječ

$$c(x) = x^6 + \alpha^4 x^3 + x^2 + \alpha^4 x + \alpha^5.$$

Provjera potvrđuje da je $c(\alpha^j) = 0$ za $j = 1, 2, 3, 4$, dakle kodna riječ je djeljiva s $g(x)$.

4.3 Izvorna konstrukcija: kôd kao vrijednosti polinoma

Uvod je kôd motivirao slikom u kojoj je kodna riječ niz vrijednosti polinoma, dok ga gornja definicija gradi preko djeljivosti s $g(x)$. To su dvije slike istog objekta i vrijedi ih spojiti.

U izvornoj konstrukciji Reeda i Solomona iz 1960. poruka je polinom f stupnja manjeg od k , a kodna riječ je niz njegovih vrijednosti u n različitih točaka polja:

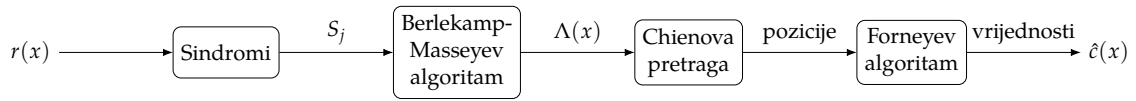
$$(f(\beta_0), f(\beta_1), \dots, f(\beta_{n-1})), \quad \beta_i \text{ međusobno različiti.}$$

Iz te slike minimalna udaljenost je gotovo očita. Dvije razne poruke daju polinome $f \neq h$, oba stupnja manjeg od k , pa je $f - h$ polinom različit od nule stupnja manjeg od k . Takav polinom ima najviše $k - 1$ nultočaka, dakle vrijednosti f i h podudaraju se na najviše $k - 1$ pozicija, a razlikuju se na barem $n - (k - 1) = n - k + 1$ njih. Udaljenost bilo koje dvije kodne riječi je stoga barem $n - k + 1$. To je isti d kao u Teoremu 1, ovdje dobiven izravno iz činjenice da polinom stupnja manjeg od k ima najviše $k - 1$ nultočaka.

Obje konstrukcije opisuju isti kôd, uz standardnu identifikaciju preko diskretne Fourierove preobrazbe nad konačnim poljem. Uvjet $c(\alpha^i) = 0$ za $i = 1, \dots, n - k$ iz (4.2) znači da spektar niza koeficijenata iščezava na $n - k$ uzastopnih mjesta, što je dualno tomu da je riječ niz vrijednosti polinoma stupnja manjeg od k . Most između dviju slika je Mattson-Solomonov polinom [2, pogl. 10]. Zato se Reed-Solomonovi kodovi istovremeno vide kao kodovi vrijednosti polinoma i kao ciklički kodovi zadani generirajućim polinomom.

5 | Dekodiranje

Dekodiranje je teži smjer. Primatelj dobiva riječ $r(x) = c(x) + e(x)$, gdje je $e(x)$ nepoznati polinom pogreške. Treba pronaći gdje su pogreške i kolike su, a da se ne zna ni koliko ih ima. Postupak ide u četiri koraka, prikazana na slici 5.1.



Slika 5.1: Tok podataka pri dekodiranju, od primljene riječi $r(x)$ do ispravljene $\hat{c}(x)$.

5.1 Sindromi

Sindrom je vrijednost primljene riječi u jednoj nultočki generirajućeg polinoma:

$$S_j = r(\alpha^j) = c(\alpha^j) + e(\alpha^j) = e(\alpha^j), \quad j = 1, \dots, n - k. \quad (5.1)$$

Kako je $c(\alpha^j) = 0$, sindrom ovisi samo o pogrešci. Ako su svi sindromi nula, primljena riječ je ispravna kodna riječ i dekodiranje staje. Inače sindromi nose podatke o pogreškama, ali u zamotanom obliku koji tek treba razriješiti.

Primjer 3. Nastavljamo primjer 2. Neka se kodna riječ ošteti na dva mjesta, s pogreškom α^2 na poziciji x^5 i pogreškom α^3 na poziciji x^2 . Primljena riječ je

$$r(x) = x^6 + \alpha^2 x^5 + \alpha^4 x^3 + \alpha^1 x^2 + \alpha^4 x + \alpha^5.$$

Uvrštavanjem prema (5.1) dobivamo sindrome

$$(S_1, S_2, S_3, S_4) = (\alpha^4, \alpha^4, \alpha^5, \alpha^2).$$

Nisu svi nula, pa riječ sadrži pogreške.

5.2 Polinom lokatora pogrešaka

Neka su pogreške na v nepoznatih pozicija. Za svaku definiramo lokator $X_l = \alpha^{i_l}$, gdje je i_l pozicija l -te pogreške. Polinom lokatora pogrešaka je

$$\Lambda(x) = \prod_{l=1}^v (1 - X_l x) = 1 + \Lambda_1 x + \dots + \Lambda_v x^v, \quad (5.2)$$

a njegove nultočke su inverzi lokatora, X_l^{-1} . Sindromi i koeficijenti od $\Lambda(x)$ povezani su Newtonovim identitetima. Uz oznaku S_j za sindrome vrijedi

$$S_r + \sum_{j=1}^v \Lambda_j S_{r-j} = 0, \quad r = v+1, v+2, \dots \quad (5.3)$$

Kad bismo znali v , ovo bi bio linearan sustav u Λ_j . Ne znamo ga, pa umjesto rješavanja fiksnog sustava tražimo $\Lambda(x)$ najmanjeg stupnja koji zadovoljava (5.3). Taj posao radi Berlekamp-Masseyev algoritam.

5.3 Berlekamp-Masseyev algoritam

Algoritam je najzahtjevniji dio dekodera i vrijedi ga vidjeti u cijelosti, a ne kao crnu kutiju. Sindrome obrađuje jedan po jedan i uz put održava trenutni lokator $\Lambda(x)$, pomoćni polinom $B(x)$ iz zadnjeg pravog popravka, trenutni broj pogrešaka L i pomak m . Na svakom koraku mjeri koliko trenutni $\Lambda(x)$ promašuje idući sindrom. Ta mjera je diskrepancija

$$\Delta_r = S_r + \sum_{j=1}^L \Lambda_j S_{r-j}. \quad (5.4)$$

Ako je $\Delta_r = 0$, lokator još slaže sve videne sindrome i mijenja se samo pomak, $m \leftarrow m+1$. Ako nije, lokator se popravljia pravilom

$$\Lambda(x) \leftarrow \Lambda(x) - \frac{\Delta_r}{b} x^m B(x), \quad (5.5)$$

gdje je b diskrepancija zapamćena u zadnjem pravom popravku. Uz to, kad je $2L \leq r-1$, stupanj lokatora mora narasti, pa se ažurira $L \leftarrow r-L$, spremi $B(x) \leftarrow \Lambda(x)$ prije popravka, $b \leftarrow \Delta_r$ i $m \leftarrow 1$. Inače stupanj ostaje isti i raste samo pomak, $m \leftarrow m+1$. Korak s uvjetom $2L \leq r-1$ je onaj u kojem algoritam zaključuje da je vidio dovoljno neslaganja da poveća procijenjeni broj pogrešaka.

Primjer 4. *Provedimo algoritam na sindromima $(S_1, S_2, S_3, S_4) = (\alpha^4, \alpha^4, \alpha^5, \alpha^2)$ iz primjera 3. Početno je $\Lambda(x) = 1$, $B(x) = 1$, $L = 0$, $b = 1$, $m = 1$. Tablica prati stanje nakon svakog sindroma.*

Provjerimo prva dva retka rukom. U koraku $r = 1$ je $\Delta_1 = S_1 = \alpha^4$, uvjet $2L = 0 \leq 0$ vrijedi, pa je $\Lambda(x) = 1 - \frac{\alpha^4}{1} x^1 \cdot 1 = \alpha^4 x + 1$ i L raste na 1. U koraku $r = 2$ je $\Delta_2 = S_2 + \Lambda_1 S_1 = \alpha^4 + \alpha^4 \cdot \alpha^4 = \alpha^2$, ali sada $2L = 2 > 1$, pa stupanj ostaje 1 i lokator postaje $x + 1$. Konačni rezultat je

$$\Lambda(x) = x^2 + \alpha^3 x + 1,$$

polinom stupnja 2, što znači dvije pogreške. To se slaže s onim što smo ubacili.

r	S_r	Δ_r	L	$\Lambda(x)$ nakon koraka
–	–	–	0	1
1	α^4	α^4	1	$\alpha^4 x + 1$
2	α^4	α^2	1	$x + 1$
3	α^5	1	2	$\alpha^3 x^2 + x + 1$
4	α^2	α^1	2	$x^2 + \alpha^3 x + 1$

Tablica 5.1: Berlekamp-Masseyev algoritam na sindromima iz primjera 3. Koraci $r = 1$ i $r = 3$ zadovoljavaju $2L \leq r - 1$ i podižu L ; koraci $r = 2$ i $r = 4$ ne mijenjaju stupanj.

5.4 Chienova pretraga

Nultočke od $\Lambda(x)$ nalazimo iscrpnom provjerom svih elemenata polja. To je Chienova pretraga. Za svaku poziciju i provjerimo je li $\Lambda(\alpha^{-i}) = 0$. Ako jest, na toj poziciji je pogreška. Polje je konačno, pa je pretraga uvijek izvediva, a u sklopovlju se izvodi vrlo brzo.

Primjer 5. Pretraga nad $\Lambda(x)$ iz primjera 4 pronalazi nultočke koje odgovaraju pozicijama x^2 i x^5 . To su točno mjesta na kojima smo unijeli pogreške.

5.5 Forneyev algoritam

Ostaje odrediti kolike su pogreške. Forneyev algoritam daje vrijednost pogreške na svakoj pronađenoj poziciji preko formule koja koristi polinom sindroma, polinom lokatora i njegovu formalnu derivaciju. Definiramo evaluator pogreške

$$\Omega(x) = S(x) \Lambda(x) \bmod x^{n-k},$$

a vrijednost pogreške na poziciji s lokatorom X_l je¹

$$e_l = \frac{\Omega(X_l^{-1})}{\Lambda'(X_l^{-1})}.$$

Kad znamo i pozicije i vrijednosti, pogrešku oduzmemo od primljene riječi i dobijemo natrag kodnu riječ.

Primjer 6. Za naš primjer evaluator ispada $\Omega(x) = \alpha^5 x + \alpha^4$. Forneyeva formula na pozicijama x^2 i x^5 daje vrijednosti α^3 odnosno α^2 . To su upravo unesene pogreške. Oduzimanjem od $r(x)$ vraćamo se na

$$c(x) = x^6 + \alpha^4 x^3 + x^2 + \alpha^4 x + \alpha^5,$$

izvornu kodnu riječ iz primjera 2. Isprava je potpun.

¹Formula u ovom obliku pretpostavlja da sindromi počinju od $j = 1$, dakle $S_j = r(\alpha^j)$. Za drukčiji početni indeks j_0 pojavljuje se dodatni faktor $X_l^{1-j_0}$, koji priložena implementacija pokriva u općenitom slučaju.

5.6 Primjer nad većim poljem

Mali primjer nad $GF(8)$ pokazuje mehaniku, ali stvarni kodovi rade nad $GF(256)$. Priložena implementacija kodira 17 podatkovnih simbola s 10 paritetnih, pa ispravlja do 5 pogrešaka. Poruka je niz znakova Reed-Solomon 1960. Nakon što se ošteti pet simbola na različitim pozicijama, dekoder pronalazi svih pet i vraća izvornu poruku bez greške.

Granica jednoznačnog dekodiranja je oštra. Uz 10 paritetnih simbola i $t = 5$, šest pogrešaka je previše da bi se jamčila jedinstvena najbliža kodna riječ, i implementacija to prijavljuje umjesto da vrati krivi rezultat. Kôd zna kad je premašen, pa se pogrešan ispravak može izbjeći. Vrijedi naglasiti što točno ta granica znači. Ona je oštra za jednoznačno dekodiranje, dakle za zahtjev da dekoder vrati jednu, sigurno najbližu kodnu riječ. Ako dopustimo da dekoder vrati kratak popis kandidata, granica se pomiče dalje, o čemu govori odjeljak 5.8.

5.7 Brisanja

Do sada smo pretpostavljali da su i pozicije i vrijednosti pogrešaka nepoznate. Često je poznato barem gdje je podatak nepouzdan. Demodulator ili unutarnji kôd znaju označiti simbol kao sumnjiv, ali bez ispravne vrijednosti. Takva pozicija zove se *brisanje* (engl. *erasure*).

Poznata pozicija je pola manje posla. Nepoznata pogreška troši dva paritetna simbola, jedan za pronalazak pozicije i jedan za vrijednost. Brisanje troši samo jedan, jer je pozicija već poznata. Zato Reed-Solomonov kôd s $n - k$ paritetnih simbola ispravlja svaki uzorak s ε brisanja i τ nepoznatih pogrešaka čim je

$$2\tau + \varepsilon \leq n - k. \quad (5.6)$$

U graničnom slučaju bez nepoznatih pogrešaka to je $n - k$ brisanja, dvostruko više nego pogrešaka. Dekodiranje se malo mijenja. Iz poznatih pozicija brisanja sagradi se njihov lokator, sindromi se njime prilagode (Forneyeva modifikacija), a na prilagođenim sindromima Berlekamp-Masseyev algoritam traži još samo nepoznate pogreške. Chien i Forney onda rade kao prije, na uniji pozicija.

5.8 Dekodiranje popisom

Klasični dekoder iz ovog poglavlja je jednoznačan. Unutar polumjera $t = \lfloor (n - k)/2 \rfloor$ oko primljene riječi leži najviše jedna kodna riječ i dekoder je vrati. Izvan tog polumjera može ih biti više ili nijedna, pa jednoznačni dekoder mora odustati. To je smisao granice iz prethodnog odjeljka.

Devedesetih se pokazalo da granica nije kraj priče. Sudan je 1997. dao algoritam koji za kodove niske stope vraća *popis* svih kodnih riječi unutar polumjera većeg od t [6]. Guruswami i Sudan poboljšali su ga 1999. do polumjera blizu Johnsonove granice, približno $n - \sqrt{nk}$, koja je za niske stope osjetno veća od t [7].

Umjesto jednog odgovora dekodier vraća kratak popis kandidata, iz kojeg se pravi bira dodatnim podatkom, primjerimice kontrolnim zbrojem.

Ideja je geometrijska. Kroz n točaka (α^i, r_i) provuče se polinom dviju varijabli $Q(x, y)$ koji u svakoj točki iščezava s dovoljnom kratnošću. Zatim se $Q(x, y)$ faktORIZIRA, a svaki činitelj oblika $y - f(x)$ s f stupnja manjeg od k daje jednog kandidata za poruku. Kratnost u interpolaciji je ono što dopušta veći polumjer nego kod jednoznačnog dekodera. Ovaj smjer je i danas aktivan, pa Reed-Solomonovi kodovi, iako opisani 1960., ostaju predmet suvremenih istraživanja.

6 | Primjene

Reed-Solomonovi kodovi su posvuda gdje podaci prolaze kroz nepouzdan kanal ili medij.

Na CD-u i DVD-u koristi se kombinacija dvaju isprepletenih Reed-Solomonovih kodova, poznata kao CIRC. Dva kôda rade u slijedu. Unutarnji kôd ispravi lakše pogreške, a teže simbole ne pogađa naslijepo nego ih označi kao brisanja i proslijedi vanjskom kôdu. Vanjski kôd te oznake iskoristi po pravilu (5.6), u kojem poznata pozicija stoji upola jeftinije od nepoznate pogreške. Isprepletanje između dvaju kôda razbija dugu grebotinu na mnogo kratkih oštećenja raspoređenih po više kodnih riječi. Zajedno to znači da disk s vidljivim ogrebotinama i dalje svira bez preskakanja. Upravo prosljeđivanje brisanja iz unutarnjeg u vanjski kôd objašnjava zašto CIRC radi bolje nego dva neovisna kôda.

QR kodovi ugrađuju Reed-Solomonov paritet u samu sliku. Ovisno o razini zaštite, ispravlja se od 7 do 30 posto oštećenih simbola. Zato QR kôd radi i kad je dio prekriven logotipom ili zamrljan.

RAID-6 sustavi za pohranu koriste Reed-Solomonovu aritmetiku za drugi paritetni disk. Time podnose ispad dvaju diskova istovremeno, bez gubitka podataka.

U svemirskim misijama kôd je bio dio komunikacije sonde Voyager, gdje se slabi signal s ruba Sunčeva sustava ispravljaao na Zemlji. Slična shema živi u standardima digitalne televizije DVB i u dubinskom svemirskom komuniciranju [3].

7 | Zaključak

Reed-Solomonovi kodovi spajaju čistu algebru s vrlo praktičnim rezultatom. Poruka postaje polinom nad konačnim poljem, paritet nastaje iz djeljivosti generirajućim polinomom, a dekodiranje kroz četiri koraka vraća izvorne podatke i onda kad je dio primljenog niza oštećen. Proveden primjer nad $GF(8)$ pokazao je cijeli put, od kodiranja preko sindroma i lokatora do konačnog ispravka dvije pogreške.

Snaga kôda leži u radu sa simbolima i u dostizanju Singletonove granice, dokazanoj u Teoremu 1, čime je optimalan u smislu razmaka među kodnim riječima. Granica jednoznačnog dekodiranja je poznata unaprijed i kôd zna kad je premašen, a dekodiranje popisom pokazuje da se i preko te granice može ići. Ta kombinacija čvrste teorije i praktične pouzdanosti objašnjava zašto je kôd iz 1960. godine i danas ugrađen u diskove, QR kodove, sustave pohrane i svemirske veze, i zašto je i dalje predmet istraživanja.

Literatura

- [1] I. S. REED, G. SOLOMON, *Polynomial Codes over Certain Finite Fields*, Journal of the Society for Industrial and Applied Mathematics **8**(1960), 300–304.
- [2] F. J. MACWILLIAMS, N. J. A. SLOANE, *The Theory of Error-Correcting Codes*, North-Holland, Amsterdam, 1977.
- [3] S. B. WICKER, V. K. BHARGAVA (UR.), *Reed-Solomon Codes and Their Applications*, IEEE Press, New York, 1994.
- [4] R. LIDL, H. NIEDERREITER, *Introduction to Finite Fields and Their Applications*, Cambridge University Press, Cambridge, 1994.
- [5] W. A. GEISEL, *Tutorial on Reed-Solomon Error Correction Coding*, NASA Technical Memorandum 102162, 1990.
- [6] M. SUDAN, *Decoding of Reed-Solomon codes beyond the error-correction bound*, Journal of Complexity **13**(1997), 180–193.
- [7] V. GURUSWAMI, M. SUDAN, *Improved decoding of Reed-Solomon and algebraic-geometry codes*, IEEE Transactions on Information Theory **45**(1999), 1757–1767.

Sažetak

Reed-Solomonovi kodovi ispravljaju pogreške nastale prijenosom podataka kroz nepouz dane kanale. Rad izlaže potrebnu algebru konačnih polja $GF(2^m)$ i polinoma nad njima, definira kôd $RS(n, k)$ i dokazuje da mu je minimalna udaljenost $d = n - k + 1$, te opisuje sistematsko kodiranje generirajućim polinomom i vezu s izvornom konstrukcijom preko vrijednosti polinoma. Dekodiranje je prikazano kroz sindrome, Berlekamp-Masseyev algoritam s tablicom iteracija, Chienovu pretragu i Forneyev algoritam, uz potpuno proveden brojčani primjer nad poljem $GF(8)$ koji ispravlja dvije pogreške. Slijede dva proširenja, dekodiranje uz brisanja i dekodiranje popisom koje prelazi klasičnu granicu ispravljlivosti. Na kraju su navedene primjene u optičkim medijima, QR kodovima, sustavima pohrane i svemirskim komunikacijama.

Ključne riječi

reed-solomonovi kodovi, ispravljanje pogrešaka, konačna polja, generirajući polinom, minimalna udaljenost, berlekamp-massey, brisanja, dekodiranje popisom, kodiranje kanala

Naslov rada na engleskom jeziku

Summary

Reed-Solomon codes correct errors introduced when data passes through unreliable channels. This paper presents the required algebra of finite fields $GF(2^m)$ and polynomials over them, defines the $RS(n, k)$ code and proves that its minimum distance is $d = n - k + 1$, and describes systematic encoding by a generator polynomial together with its link to the original evaluation construction. Decoding is shown through syndromes, the Berlekamp-Massey algorithm with an iteration table, the Chien search, and the Forney algorithm, with a fully worked numerical example over $GF(8)$ that corrects two errors. Two extensions follow, erasure decoding and list decoding beyond the classical error-correction bound. Applications in optical media, QR codes, storage systems, and space communication close the paper.

Keywords

reed-solomon codes, error correction, finite fields, generator polynomial, minimum distance, berlekamp-massey, erasures, list decoding, channel coding